



Universidad Nacional del Comahue
Facultad de Informática
Departamento Ingeniería en Sistemas

Lenguajes de consulta relacionales comerciales

Gestión de Bases de Datos

Licenciatura en Ciencias de la Computación
3° Año

Integrantes del Grupo:

- Agustin Ferreyra – Legajo: 4025
- Milagros Ruiz – Legajo: 4564

Docentes:

- Prof. Titular: Agustina Buccella
- Ayudante: Ignacio San Pedro Delgado

Fecha de entrega:

17/11/2025

Lenguajes de consulta relacionales comerciales	1
Unidad VI	3
Investigación	3
Índices	3
Errores comunes	3
Buenas prácticas	3
Índices en PostgreSQL	4
Btree	4
Hash	4
Tabla comparativa	5
VACUUM	6
ANALYZE	6
EXPLAIN	6
Usando índices en PostgreSQL	7
Comportamiento del índice Hash:	10
Comportamiento del índice B-tree Multicolumna	12
Comportamiento del Índice B-tree Parcial	14
Desnormalizando....	17
MER luego de la desnormalización:	18
Consulta:	18
Tiempos antes de desnormalizar	19
Modificaciones para lograr la desnormalización	19
Tiempos luego de desnormalizar	22
Participaciones TPO2	23
Grupal	23
Ruiz Milagros	23
Ferreyra Agustín	23
Bibliografía	24

Unidad VI

Investigación

1. Buscar información sobre uso de índices, buenas prácticas y errores típicos.

Índices

Un índice de base de datos es una estructura de datos que almacena un subconjunto de las columnas y filas de una tabla ~~en un orden ordenado~~ de forma ordenada. El índice actúa como un puntero a la tabla original, lo que permite al motor de base de datos encontrar y recuperar los datos más rápido que recorrer toda la tabla.^[1]

Los índices se utilizan para:

- Mejorar el rendimiento. En la mayoría de los casos, el acceso a los datos es más rápido con un índice. Aunque no puede crearse un índice para una vista, un índice creado para la tabla en la que se basa una vista puede mejorar a veces el rendimiento de las operaciones en esta vista.
- Asegurar la exclusividad. Una tabla con un índice de unicidad no puede tener filas con claves idénticas.

Errores comunes

Los índices de bases de datos vienen con ciertas compensaciones e inconvenientes que deben tenerse en cuenta antes de crearlos y usarlos. Estos incluyen el espacio que consumen, lo que puede afectar su capacidad y costo de almacenamiento. Cuantas más columnas y filas incluya en un índice, más espacio ocupará; Los índices de texto completo, por ejemplo, pueden ser muy grandes y complejos. Además, los índices deben actualizarse siempre que se inserten, actualicen o eliminen datos de la tabla, lo que puede afectar al rendimiento de las operaciones de escritura. Con el tiempo, los índices pueden fragmentarse, lo que lleva a una disminución del rendimiento de las operaciones de lectura. Como resultado, es necesario el monitoreo y mantenimiento regular de los índices. Además, los índices de bases de datos agregan complejidad y sobrecarga al diseño y la administración de su base de datos. Debe elegir el tipo, el tamaño y el número de índices correctos para las tablas en función de las características de los datos y los patrones de consulta; También es importante probar y medir el impacto de dichos índices en el rendimiento de las consultas.^[2]

Buenas prácticas

Para aprovechar al máximo la indización de bases de datos, es importante seguir los procedimientos recomendados y las directrices que pueden ayudarle a crear y usar índices de manera eficaz y eficiente. El primer paso es analizar los datos y las consultas: puede usar herramientas como el optimizador de consultas, el plan de ejecución o las estadísticas de uso del índice para identificar las columnas y tablas que pueden beneficiarse de la indicación, así como el tipo y el orden de las columnas de índice. Al crear un índice, elija columnas selectivas y relevantes con un alto grado de singularidad y diversidad en sus valores, así como las que se usan con frecuencia en las condiciones, uniones u ordenaciones de consulta. Además, evite índices innecesarios y redundantes que rara vez o nunca son utilizados por sus consultas o duplicados por otros índices. El Asistente para ajuste de índices, el asesor de índices o la utilidad de limpieza de índices pueden ayudarle a buscar y quitar índices duplicados o no utilizados.^[3]

2. Investigar los tipos de índices que provee PostgreSQL con las formas de crearlos. Centrarse en btree y hash.

Índices en PostgreSQL

PostgreSQL proporciona varios tipos de índices: árbol B, Hash, GiST, SP-GiST, GIN, BRIN y la extensión bloom. Cada tipo de índice utiliza un algoritmo diferente que se adapta mejor a diferentes tipos de cláusulas indexables.

Por defecto, el `CREATE INDEX` El comando crea índices del tipo btree, que se ajustan a las situaciones más comunes. [\[4\]](#)

Btree

El índice B-Tree es el tipo de índice principal que utiliza PostgreSQL. Permite acelerar consultas que involucran comparaciones de igualdad y de rango, porque mantiene los valores almacenados en un orden lógico. El planificador utiliza un índice B-Tree cuando la columna indexada aparece en condiciones que usan operadores como: `<`, `<=`, `=`, `>=`, `>`, `BETWEEN`, `IN`, o cuando se necesita ordenar mediante `ORDER BY`, ya que el índice se encuentra físicamente ordenado.

Los índices B-Tree pueden ser de varias formas:

- De una sola columna: optimizan búsquedas simples por igualdad, rango u ordenamiento sobre ese atributo.
- Multicolumna (n columnas): el orden de las columnas es importante; el índice solo puede usarse si la consulta filtra en el mismo orden en que fueron declaradas las columnas dentro del índice. También permiten optimizar órdenes que coincidan con ese ordenamiento.
- Índices B-Tree `UNIQUE`: además de acelerar la búsqueda, garantizan que no existan valores duplicados en la columna o grupo de columnas indexadas.
- Índices B-Tree parciales: solamente indexan las filas que cumplen una condición específica. Son útiles para consultas repetidas sobre un subconjunto de la tabla, y reducen espacio y costos de mantenimiento.

Construcciones equivalentes a combinaciones de estos operadores, tales como `BETWEEN` y `IN`, también se puede implementar con una búsqueda de índice de árbol B. Además, un `IS NULL` o `IS NOT NULL` La condición en una columna de índice se puede utilizar con un índice de árbol B. [\[5\]](#)

Hash

PostgreSQL incluye una implementación de índices hash persistentes en el disco, que son totalmente recuperables en caso de falla. Cualquier tipo de datos puede indexarse mediante un índice hash, incluidos los tipos de datos que no tienen un orden lineal bien definido. Los índices hash almacenan solo el valor hash de los datos que se indexan, por lo que no existen restricciones sobre el tamaño de la columna de datos que se indexa.

Los índices hash solo admiten índices de una sola columna y no permiten la verificación de unicidad.

Los índices hash solo admiten el operador `=`, por lo que las cláusulas `WHERE` que especifican operaciones de rango no podrán aprovechar los índices hash.

Cada tupla de índice hash almacena sólo el valor hash de 4 bytes, no el valor real de la columna.

Como resultado, los índices hash pueden ser mucho más pequeños que los btree al indexar elementos de datos más largos, como `UUID`, `URL`, etc. La ausencia del valor de la columna también hace que

todos los escaneos de índices hash tengan pérdidas. Los índices hash pueden participar en escaneos de índices de mapas de bits y escaneos hacia atrás. [\[6\]](#)

3. Comparar los índices de postgresQL con los vistos en clase. Haga una tabla comparativa.

Tabla comparativa

Criterios de Comparación	Índices vistos en clase		Índices que provee Postgree	
	Índice de Árbol B+	Índice Asociativo	B-tree	Hash
Tipo de Acceso	Adecuado para búsquedas de rango y secuenciales	Diseñado para búsquedas de igualdad, utilizando una función de dispersión	Optimiza consultas de igualdad y rango como <, <=, =, =>, >	Optimiza búsquedas de igualdad
Tiempo de Acceso	Mantiene un rendimiento eficiente incluso al aumentar el tamaño del archivo	Rápido para búsquedas de igualdad	Rápido para consultas de igualdad y rango; es más lento con datos balanceados	Muy rápido para consultas de igualdad pero no es útil para rangos
Tiempo de Inserción	Eficiente debido al balance del árbol	Puede ser menos eficiente si hay muchas colisiones	Moderado para consultas de inserción; se mantiene eficiente porque el árbol permanece balanceado	Rápido pero puede requerir reorganización si hay colisiones de hash
Tiempo de Borrado	Similar al tiempo de inserción; mantiene el árbol balanceado	Eficiente en ausencia de colisiones	Moderado ya que puede requerir volver a organizar el árbol	Rápido pero puede ser costoso si hay colisiones frecuentes
Espacio Requerido Adicional	Requiere espacio	Moderado, almacenamiento	Moderado depende del	Moderado; almacenamiento

	adicional para nodos del árbol	adicional para tablas hash y manejo de colisiones	número de niveles del árbol	adicional para tablas de hash
--	-----------------------------------	--	-----------------------------------	----------------------------------

Tabla 32: Tabla Comparativa de Índices

4. Para PostgreSQL investigue las formas de examinar el uso de los índices (comandos EXPLAIN, VACUUM, ANALYZE).

VACUUM

Vacuum se encarga de recuperar el almacenamiento ocupado por tuplas muertas. En una operación normal de PostgreSQL las tuplas que son eliminadas o quedan obsoletas por un Update no son físicamente removidas de la tabla, quedan presentes hasta que se realiza un VACUUM.

```
VACUUM [ ( option [, ...] ) ] [ table_and_columns [, ...] ]
```

Dónde [option] indica la tarea a realizar. La opción más avarcata es FULL. Dicha opción bloquea la tabla a la que le realizamos el Vacuum y libera el máximo posible de almacenamiento de la misma con la desventaja de que lleva mucho tiempo en ejecutarse.

[table_and_columns] indica a qué tabla y columnas de la misma se aplicará el Vacuum. En caso de no seleccionar ninguna tabla se aplica a todas las tablas de las que posea permiso para realizar el Vacuum [\[7\]](#)

ANALYZE

ANALYZE recolecta estadísticas acerca del contenido de una tabla en la base de datos y guarda los resultados en un catálogo de sistema llamado pg_statistic. Esta información es usada luego para ayudar a determinar la ejecución más eficiente de ciertas consultas.

Para poder utilizar el comando Analyze uno debe poseer el privilegio de Mantener de una tabla.

Mientras que el dueño de la BD puede realizarlo sobre cualquier tabla de la misma.

```
ANALYZE [ ( option [, ...] ) ] [ table_and_columns [, ...] ]
```

donde (option) puede ser VERBOSE [boolean] | SKIP_LOCKED [boolean] |

BUFFER_USAGE_LIMIT size

Y [table_and_columns] es la tabla y columnas de la misma a la que se aplica. Si esto se deja vacío se ejecuta sobre todas las tablas y vistas materializadas de la BD para la cuales el usuario tiene permiso para analizar.

se puede realizar la siguiente puede VACUUM ANALYZE realiza un Vacuum y luego un Analyze para cada tabla seleccionada. Esta combinación es útil para rutinas de mantenimiento. [\[8\]](#)

EXPLAIN

Este comando muestra el plan de ejecución que el planificador de PostgreSQL genera para la instrucción proporcionada. El plan de ejecución muestra cómo se examinarán las tablas a las que hace referencia la instrucción (mediante un escaneo secuencial simple, un escaneo de índice, etc.) y, si se hace referencia a varias tablas, qué algoritmos de unión se utilizarán para combinar las filas necesarias de cada tabla de entrada.

EXPLAIN [(option [, ...])] statement

La parte más importante de la pantalla es el coste estimado de ejecución de la instrucción, que es la estimación del planificador sobre cuánto tiempo tardará en ejecutarse (medido en unidades de coste arbitrarias, pero que convencionalmente representan accesos a páginas de disco). En realidad, se muestran dos valores: el coste inicial antes de que se pueda devolver la primera fila y el coste total para devolver todas las filas. Para la mayoría de las consultas, lo que importa es el coste total, pero en contextos como una subconsulta en EXISTS, el planificador elegirá el menor coste inicial en lugar del menor coste total (ya que el ejecutor se detendrá después de obtener una fila). Además, si se limita el

número de filas a devolver con una cláusula LIMIT, el planificador realiza una interpolación adecuada entre los costes de los puntos finales para estimar qué plan es realmente el más económico. [\[9\]](#)

Usando índices en PostgreSQL

5. Dada la BD creada en el TPO1, definir al menos 10 instrucciones para crear índices que involucren diferentes formas de crear índices btree (simple-múltiple-columnas, ORDER BY, UNIQUE, PARCIALES) y hash. Justificar cada creación según requerimientos de la BD.

Índice Hash

Como el índice hash se utiliza para la búsqueda de valores exactos, decidimos emplearlo en la tabla obra social mediante su nombre, ya que es frecuente realizar consultas sobre las mismas mediante su nombre

```
CREATE INDEX dniPilotos ON Piloto USING HASH (dni)

CREATE INDEX nombreObraFiltro ON ObraSocial USING HASH(nombre)
```

También se utilizó índice hash para la búsqueda de los empleados a través de su nombre.

```
CREATE INDEX dniMecanicos ON Mecanico USING HASH (dni)

CREATE INDEX nombreEmpleadoFiltro ON Empleado USING HASH(nombre)
```

También decidimos emplearlo en la columna teléfonos de propietarios ya que es frecuente realizar consultas exactas sobre los teléfonos de los propietarios.

```
CREATE INDEX telefonosPropietarios ON Telefonos USING HASH (telefono)
```

Índice B-tree Simple

~~Utilizamos un B-tree simple para optimizar la consulta de el nombre único de una corporación. Este nombre lo necesitamos para saber quien es el propietario de un avión. Lo mismo realizamos para la tabla persona física con la columna dni.~~

Utilizamos un B-tree simple para optimizar la consulta de la fecha de compra de un avión. Dicha fecha es útil a la hora de realizar un mantenimiento ya que nos da un indicio de los próximos mantenimientos a realizar

```
CREATE INDEX filtro_nombre_unico ON corporacion(nombre_unico)
CREATE INDEX filtro_fecha_compra ON avion(fecha_compra)
```

Utilizamos B-tree simple para optimizar la búsqueda de los propietarios del tipo persona física, cuya consulta se realiza frecuentemente en nuestra empresa

```
CREATE INDEX filtro_nombre_persona_fisica ON personafisica(nombre);
```


Índice B-tree UNIQUE

Este índice es útil cuando queremos asegurar la unicidad en una columna o conjunto de columnas, como en el caso de claves candidatas o para forzar que un atributo no se repita, por esta razón, se decidió emplear este tipo de índice en la tabla de propietario para que no se repita el correo y para poder buscar los correos de los propietarios eficientemente.

```
CREATE UNIQUE INDEX correo_propietario ON propietario(correo)
```

Índice B-tree Parcial

Dado que permite indexar sólo una parte de los datos, decidimos implementarlo en la columna capacidad de la tabla Avión, filtrando únicamente por los aviones con capacidad mayor a 300, ya que a partir de esta capacidad ya son aviones internacionales

```
CREATE INDEX idx_avion_pesado ON Avion(capacidad) WHERE capacidad > 3000;
```

A su vez también se decidió crear otro índice B-tree parcial con el mismo propósito para filtrar los aviones con capacidad ≤ 300 .

```
CREATE INDEX idx_avion_liviano ON Avion(capacidad) WHERE capacidad < 3000;
```

Índice B-tree con ORDER BY

Dado que este índice es ideal cuando las consultas requieren ordenar los resultados, se decidió utilizar en la columna fecha fin junto con fecha inicio, dni de la persona y id del hangar de los últimos alquileres. Esto porque el aeropuerto necesita frecuentemente los resultados de los últimos alquileres.

```
CREATE INDEX alquileresPorUltimaFecha ON Alquila (fecha_fin DESC, fecha_Inicio, dni,id_hangar)
```

Índice B-tree Multicolumna

Btree con multi columnas se lo aplicamos a la tabla mantenimiento a las columnas fecha y tipo. Esto lo realizamos con el objetivo de agilizar las consultas en las que queremos encontrar los mantenimientos de cierto tipo en una fecha determinada.

```
CREATE INDEX tipo_y_fecha_mantenimiento ON mantenimiento (tipo, fecha);
```


6. Insertar a las tablas involucradas muchas tuplas (posiblemente más de 100mil) de forma de EXAMINAR su comportamiento con los índices creados. Los deben hacer para sólo 4 de las instrucciones de índices definidas anteriormente que involucren hash y btree.

Insertamos 100.000 mecánicos con la siguiente consulta:

```
INSERT INTO mecánico ( dni, nombre, apellido, fecha_nacimiento, cuil, salario )
SELECT
  30000000 + i AS dni,
  'Nombre_' || i AS nombre,
  'Apellido_' || i AS apellido,
  DATE '1986-03-11' AS fecha_nacimiento,
  20111111111 AS cuil,
  2000000 AS salario
FROM generate_series(1, 100000) AS g(i);
```

Insertamos 100.000 aviones con la siguiente consulta:

```
INSERT INTO avión(id_avion, tipo, capacidad,peso,fecha_compra)
SELECT
  i AS id_avion,
  'comercial' AS tipo,
  300 AS capacidad,
  500 AS peso,
  DATE '2015-05-28' AS fecha_compra
FROM generate_series(6, 100006) AS g(i);
```

Insertamos 100.000 mantenimientos con la siguiente consulta:

```
INSERT INTO mantenimiento(fecha,id_avion, tipo, horas )
SELECT
  DATE '2015-05-28' AS fecha,
  i AS id_avion,
  'fuselaje' AS tipo,
  5 AS horas
FROM generate_series(3, 100000) AS g(i);
```

Insertamos 100.000 tuplas de la tabla realiza con la siguiente consulta:

```
INSERT INTO realiza(dni,fecha,id_avion )
SELECT
  30000001+i AS dni,
  DATE '2015-05-28' AS fecha,
  1+i AS id_avion
FROM generate_series(2, 99999) AS g(i);
```

7. Comparar los resultados del EXAMINAR anterior contra el mismo EXAMINAR pero sin índices creados . Realice una tabla comparativa.

Comportamiento del índice Hash:

Tiempo de inserción:

Consulta:

```
EXPLAIN ANALYZE
INSERT INTO mecanico (dni, nombre, apellido, fecha_nacimiento, cuil)
VALUES(30000000,'juan','perez','01-05-2000',20111111111);
```

Comportamiento con el índice aplicado:

QUERY PLAN
Insert on mecanico (cost=0.00..0.01 rows=1 width=270) (actual time=0.156..0.156 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=270) (actual time=0.011..0.011 rows=1 loops=1)
Planning time: 0.024 ms
Trigger for constraint mecanico_cuil_fkey: time=0.371 calls=1
Execution time: 0.681 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Insert on mecanico (cost=0.00..0.01 rows=1 width=270) (actual time=0.075..0.075 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=270) (actual time=0.006..0.006 rows=1 loops=1)
Planning time: 0.033 ms
Trigger for constraint mecanico_cuil_fkey: time=0.355 calls=1
Execution time: 0.572 ms

A la hora de insertar una tupla el tiempo cuando tenemos el índice es mayor, esto sucede ya que no requiere actualizar la estructura adicional del índice.

Tiempo con índice	Tiempo sin índice
Execution time: 0.572 ms	Execution time: 0.681 ms

Tiempo de borrado:

Consulta:

```
EXPLAIN ANALYZE
```

```
DELETE FROM mecanico
WHERE dni=30000000;
```

Comportamiento con el índice aplicado:

QUERY PLAN
Delete on mecanico (cost=0.00..8.02 rows=1 width=6) (actual time=0.050..0.050 rows=0 loops=1)
-> Index Scan using dnimecanicos on mecanico (cost=0.00..8.02 rows=1 width=6) (actual time=0.028..0.029 rows=1 loops=1)
Index Cond: ((dni)::integer = 25000000)
Planning time: 0.136 ms
Trigger for constraint realiza_dni_fkey: time=0.342 calls=1
Execution time: 0.392 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Delete on mecanico (cost=0.29..8.31 rows=1 width=6) (actual time=0.030..0.030 rows=0 loops=1)
-> Index Scan using mecanico_pkey on mecanico (cost=0.29..8.31 rows=1 width=6) (actual time=0.008..0.009 rows=1 loops=1)
Index Cond: ((dni)::integer = 25000001)
Planning time: 0.116 ms
Trigger for constraint realiza_dni_fkey: time=0.336 calls=1
Execution time: 0.510 ms

Podemos observar que a la hora de borrar tuplas no existe una mejora significativa en el tiempo de ejecución cuando existe el índice comparado cuando no se encuentra.

Tiempo con índice	Tiempo sin índice
Execution time: 0.510 ms	Execution time: 0.392 ms

Tiempo de búsqueda:

Consulta:

```
EXPLAIN ANALYZE
SELECT m.nombre, m.apellido, m.dni
FROM mecanico m
WHERE m.dni >= 30050000
```

Comportamiento con el índice aplicado:

QUERY PLAN
Index Scan using mecanico_pkey on mecanico (cost=0.29..2561.89 rows=49728 width=30) (actual time=0.021..7.658 rows=50005 loops=1)
Index Cond: ((dni)::integer > 30050000)
Planning time: 0.147 ms
Execution time: 8.904 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Index Scan using mecanico_pkey on mecanico (cost=0.29..2561.91 rows=49729 width=30) (actual time=0.018..7.983 rows=50005 loops=1)
Index Cond: ((dni)::integer > 30050000)
Planning time: 0.110 ms
Execution time: 9.239 ms

Podemos observar que existe una pequeña mejora cuando realizamos la búsqueda con el índice aplicado. Esta mejora puede parecer pequeña pero en una BD con millones de tuplas puede marcar la diferencia.

Tiempo con índice	Tiempo sin índice
Execution time: 8.904 ms	Execution time: 9.239 ms

Comportamiento del índice B-tree Multicolumna

Tiempo de insertion:

Consulta:

```
EXPLAIN ANALYZE
INSERT INTO mantenimiento(id_avion,tipo, horas,fecha)
values(40000,'comercial',5,'01-05-2000');
```

Comportamiento con el índice aplicado:

QUERY PLAN
Insert on mantenimiento (cost=0.00..0.01 rows=1 width=44) (actual time=0.096..0.096 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=44) (actual time=0.010..0.011 rows=1 loops=1)
Planning time: 0.043 ms
Trigger for constraint mantenimiento_id_avion_fkey: time=0.351 calls=1
Execution time: 0.562 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Insert on mantenimiento (cost=0.00..0.01 rows=1 width=44) (actual time=0.068..0.068 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=44) (actual time=0.009..0.009 rows=1 loops=1)
Planning time: 0.038 ms
Trigger for constraint mantenimiento_id_avion_fkey: time=0.351 calls=1
Execution time: 0.519 ms

Observamos una mejora a la hora de insertar una tupla con el índice aplicado.

Tiempo con índice	Tiempo sin índice
Execution time: 0.519 ms	Execution time: 0.562 ms

Tiempo de borrado

Consulta:

```
EXPLAIN ANALYZE
DELETE FROM mantenimiento
WHERE id_avion=40000;
```

Comportamiento con el índice aplicado:

QUERY PLAN
Delete on mantenimiento (cost=0.29..6.06 rows=1 width=6) (actual time=0.043..0.043 rows=0 loops=1)
-> Index Scan using mantenimiento_pkey on mantenimiento (cost=0.29..6.06 rows=1 width=6) (actual time=0.018..0.019 rows=1 loops=1)
Index Cond: (fecha = '2000-02-02'::date)
Planning time: 0.140 ms
Trigger for constraint realiza_id_avion_fkey: time=11.729 calls=1
Execution time: 8.158 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Delete on mantenimiento (cost=0.29..6.06 rows=1 width=6) (actual time=0.044..0.044 rows=0 loops=1)
-> Index Scan using mantenimiento_pkey on mantenimiento (cost=0.29..6.06 rows=1 width=6) (actual time=0.027..0.027 rows=1 loops=1)
Index Cond: (fecha = '2000-02-02'::date)

Planning time: 0.107 ms
Trigger for constraint realiza_id_avion_fkey: time=8.100 calls=1
Execution time: 11.791 ms

Observamos una mejora significativa a la hora de borrar tuplas con el índice aplicado

Tiempo con índice	Tiempo sin índice
Execution time: 8.158 ms	Execution time: 11.791 ms

Tiempo de búsqueda

Consulta:

```
EXPLAIN ANALYZE
SELECT m.id_avion, fecha
FROM mantenimiento m
WHERE m.id_avion >= 40000 AND m.id_avion < 60000 ;
```

Comportamiento con el índice aplicado:

QUERY PLAN
Seq Scan on mantenimiento (cost=0.00..2137.03 rows=19713 width=8) (actual time=3.376..9.455 rows=19999 loops=1)
Filter: ((id_avion > 40000) AND (id_avion < 60000))
Rows Removed by Filter: 80003
Planning time: 0.139 ms
Execution time: 9.996 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Seq Scan on mantenimiento (cost=0.00..2137.03 rows=19713 width=8) (actual time=2.855..9.016 rows=19999 loops=1)
Filter: ((id_avion > 40000) AND (id_avion < 60000))
Rows Removed by Filter: 80003
Planning time: 0.100 ms
Execution time: 9.526 ms

Observamos una pequeña desmejora al realizar una búsqueda con el índice aplicado

Tiempo con índice	Tiempo sin índice
Execution time: 9.996 ms	Execution time: 9.526 ms

Ya que

Comportamiento del Índice B-tree Parcial

Tiempo de inserción

Consulta:

```
EXPLAIN ANALYZE
INSERT INTO avion(id_avion, capacidad,peso,fecha_compra)
values(200000,300,400,'01-05-2000');
```

Comportamiento con el índice aplicado:

QUERY PLAN
Insert on avion (cost=0.00..0.01 rows=1 width=82) (actual time=0.091..0.091 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=82) (actual time=0.001..0.001 rows=1 loops=1)
Planning time: 0.024 ms
Trigger for constraint avion_id_propietario_fkey: time=0.078 calls=1
Execution time: 0.259 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Insert on avion (cost=0.00..0.01 rows=1 width=82) (actual time=0.071..0.071 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=82) (actual time=0.000..0.000 rows=1 loops=1)
Planning time: 0.027 ms
Trigger for constraint avion_id_propietario_fkey: time=0.064 calls=1
Execution time: 0.167 ms

Podemos observar que existe una desmejora importante a la hora de aplicar el índice. Este índice es poco efectivo a la hora de insertar tuplas.

Tiempo con índice	Tiempo sin índice
Execution time: 0.259 ms	Execution time: 0.167 ms

Tiempo de borrado

Consulta:

```
EXPLAIN ANALYZE
DELETE FROM avion
WHERE id_avion=200000;
```

Comportamiento con el índice aplicado:

QUERY PLAN
Delete on avion (cost=0.29..8.31 rows=1 width=6) (actual time=0.046..0.046 rows=0 loops=1)
-> Index Scan using avion_pkey on avion (cost=0.29..8.31 rows=1 width=6) (actual time=0.025..0.025 rows=1 loops=1)
Index Cond: (id_avion = 2)
Planning time: 0.198 ms
Trigger for constraint mantenimiento_id_avion_fkey: time=7.667 calls=1
Trigger for constraint guarda_id_avion_fkey: time=0.228 calls=1
Execution time: 7.957 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Delete on avion (cost=0.29..8.31 rows=1 width=6) (actual time=0.044..0.044 rows=0 loops=1)
-> Index Scan using avion_pkey on avion (cost=0.29..8.31 rows=1 width=6) (actual time=0.027..0.027 rows=1 loops=1)
Index Cond: (id_avion = 2)
Planning time: 0.092 ms
Trigger for constraint mantenimiento_id_avion_fkey: time=7.914 calls=1
Trigger for constraint guarda_id_avion_fkey: time=0.232 calls=1
Execution time: 8.205 ms

Observamos una pequeña desmejora a la hora de borrar una tupla con el índice aplicado.

Tiempo con índice	Tiempo sin índice
Execution time: 7.957 ms	Execution time: 8.205 ms

Tiempo de búsqueda

Consulta:

```
EXPLAIN ANALYZE
SELECT a.tipo, a.id_avion, a.peso
FROM avion a
WHERE a.id_avion>40000 AND a.id_avion<60000 ;
```

Comportamiento con el índice aplicado:

QUERY PLAN
Index Scan using avion_pkey on avion (cost=0.29..746.15 rows=19793 width=6) (actual time=0.021..3.061 rows=19999 loops=1)
Index Cond: ((id_avion > 40000) AND (id_avion < 60000))
Planning time: 0.179 ms
Execution time: 3.598 ms

Comportamiento sin el índice aplicado:

QUERY PLAN
Index Scan using avion_pkey on avion (cost=0.29..746.15 rows=19793 width=6) (actual time=0.026..2.976 rows=19999 loops=1)
Index Cond: ((id_avion > 40000) AND (id_avion < 60000))
Planning time: 0.103 ms
Execution time: 3.443 ms

Observamos una leve desmejora a la hora de realizar una búsqueda con el índice aplicado

Tiempo con índice	Tiempo sin índice
Execution time: 3.598 ms	Execution time: 3.443 ms

El comportamiento de aplicar el índice B-tree parcial no mejora el rendimiento de la búsqueda, inserción y borrado por lo que podemos concluir que dicho índice no es efectivo.

Desnormalizando....

Usando la desnormalización como otra estrategia de tuning, elegir entre 3 o 4 tablas para desnormalizar y realizar:

- Justificar la elección de dichas tablas.
- Graficar el MER resultante y su derivación a tablas.
- Demostrar la desnormalización. Qué FN tenía antes el MER y qué FN se cumple ahora en la modificación?
- Antes de hacer los cambios, analizar una consulta de las tablas que luego van a unir (una consulta con JOINS) y guardar los tiempos.
- Ejecutar los comandos SQL necesarios para transformar la BD ya creada en el TPO1

Hangar + Avión

Para optimizar las consultas más frecuentes del aeropuerto. Por ello, se requiere almacenar directamente en la tabla Avión algunos atributos del hangar donde se encuentra ya que en el sistema es muy frecuente acceder a información como la ubicación física de un avión, su hangar asignado y la capacidad del hangar.

AviónEnHangar(id_avion, tipo, capacidad, peso, fecha_compra, id_hangar, capacidad_hangar, ubicacion_hangar, fecha_ingreso, fecha_salida)

Análisis antes de la Desnormalización

Avion(id_avion, tipo, capacidad, peso, fecha_compra)

DF: FA = {id_avion → tipo, capacidad, peso, fecha_compra, id_hangar}

Guarda(id_avion, id_hangar, fecha_ingreso, fecha_salida)

Hangar(id_hangar, capacidad, ubicacion)

DF: FH = {id_hangar → capacidad, ubicacion}

En ambos casos:

Todos los atributos **dependen** de la clave entonces se encuentran en FNBC, ya que en toda DF cada atributo no primo depende únicamente de la clave.

Análisis antes de la Desnormalización

Tenemos nuevas dependencias funcionales

G = {
id_avion → tipo, capacidad, peso, fecha compra, id hangar
id_hangar → capacidad_hangar, ubicación_hangar
id_avion, id_hangar, fecha_ingreso → fecha_salida }

Este nuevo esquema nos determina que la clave primaria está compuesta por los atributos por id_hangar, id_avion y fecha_ingreso, estos serán los atributos primos y el resto no primos.

id_avion → tipo, capacidad, peso, fecha compra, id hangar → está en 1FN ya que un atributo primo depende parcialmente de una clave

id_hangar → capacidad_hangar, ubicación_hangar → está en 1FN ya que un atributo no primo depende transivamente de parte de una clave

id_avion, id_hangar, fecha_ingreso → fecha_salida → está en FNBC ya que depende totalmente de la clave

Por lo tanto el esquema nuevo, se encuentra en 1FN.

MER luego de la desnormalización:

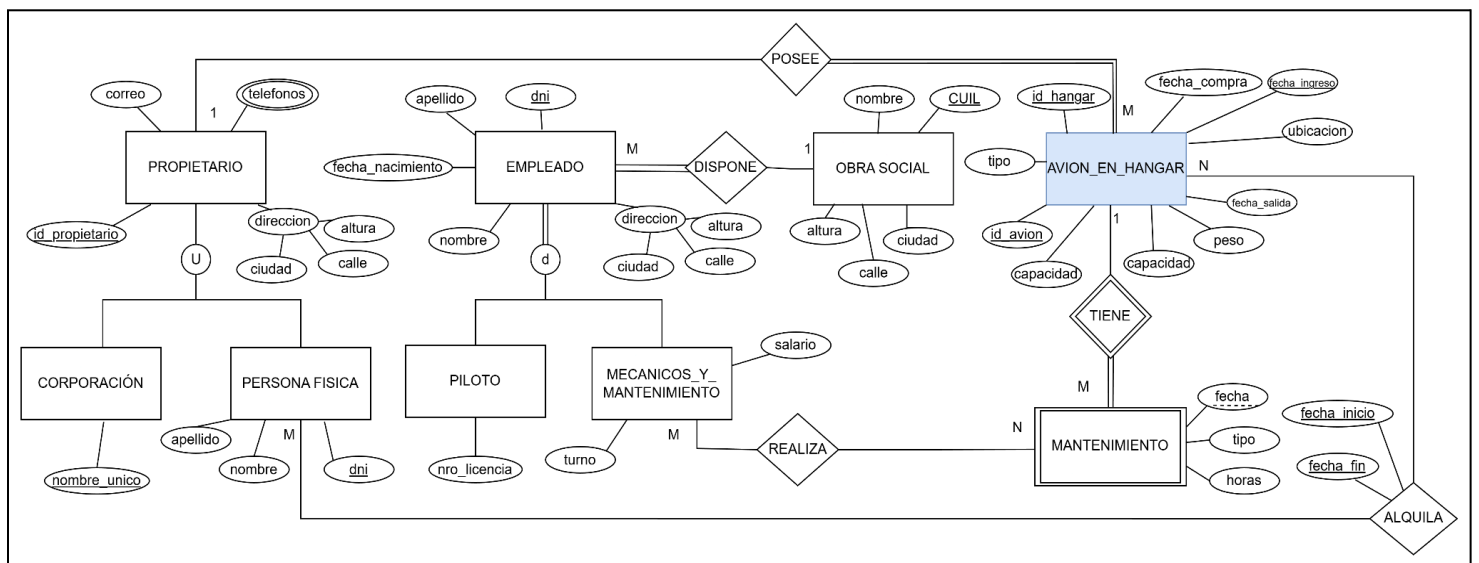


Figura 32: Mer resultante

Consulta:

Listar de los aviones de tipo comercial que hayan sido guardados en el año 2025 y del hangar en el que fueron guardados listar únicamente los que tienen ubicación en zona sur.

```
SELECT a.id_avion, a.tipo, a.id_propietario, a.peso, a.capacidad, g.fecha_ingreso, g.fecha_salida, h.id_hangar,
h.ubicacion
FROM avion a
JOIN guarda g ON a.id_avion = g.id_avion
JOIN hangar h ON g.id_hangar = h.id_hangar
WHERE g.fecha_ingreso >= DATE '2025-01-01'
AND g.fecha_ingreso < DATE '2026-01-01'
AND h.ubicacion ILIKE 'Zona Sur'
```

Tiempos antes de desnormalizar

QUERY PLAN
Nested Loop (cost=17.18..2492.29 rows=181 width=92) (actual time=0.064..79.112 rows=59990 loops=1)
Método de join, recorre una tabla y por cada fila busca coincidencias en la otra. Indica costos filas y ancho de filas estimados. luego datos reales; el tiempo bucles y cantidad de filas devueltas
-> Hash Join (cost=16.89..2427.73 rows=181 width=74) (actual time=0.060..20.816 rows=59990 loops=1)
Hash Cond: (g.id_hangar = h.id_hangar)
Este es el join interno del Nested Loop. Indica costos filas y ancho de filas estimados. luego datos reales; el tiempo bucles y cantidad de filas devueltas
-> Seq Scan on guarda g (cost=0.00..2035.23 rows=99682 width=16) (actual time=0.018..11.562 rows=59994 loops=1)
Recorre la tabla guarda completa
Filter: ((fecha_ingreso >= '2025-01-01'::date) AND (fecha_ingreso < '2026-01-01'::date))
Rows Removed by Filter: 39901
Filas leídas y descartadas por el filtro
-> Hash (cost=16.88..16.88 rows=1 width=62) (actual time=0.019..0.019 rows=4 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 9kB
Construye una tabla hash con el resultado anterior para unirla con guarda más rápidamente
-> Seq Scan on hangar h (cost=0.00..16.88 rows=1 width=62) (actual time=0.013..0.014 rows=4 loops=1)
Filter: ((ubicacion)::text ~* 'Zona Sur'::text)
Rows Removed by Filter: 4
Filas removidas de hangar según el filtro indicado
-> Index Scan using avion_pkey on avion a (cost=0.29..0.35 rows=1 width=22) (actual time=0.001..0.001 rows=1 loops=59990)
Index Cond: (id_avion = g.id_avion)
Por cada fila de guarda, busca el avión correspondiente. Se ejecutó 59.990 veces, una por cada fila del Nested Loop
Planning time: 0.542 ms
Tiempo del optimizador
Execution time: 81.157 ms
Tiempo real de ejecución

Antes de la desnormalización, la consulta que realiza un JOIN entre las tablas avión, hangar y guarda muestra un tiempo de ejecución significativo de 81.157 ms.

Modificaciones para lograr la desnormalización

Primero agregamos las columnas de la tabla hangar y la tabla guarda a la tabla avión

```
ALTER TABLE "avion"  
ADD "id_hangar" integer NOT NULL DEFAULT '0',  
ADD "capacidad_hangar" character varying(20) NULL,  
ADD "ubicacion" character varying(20) NULL,  
ADD "fecha_ingreso" date NOT NULL DEFAULT '01-01-2000',  
ADD "fecha_salida" date NOT NULL DEFAULT '01-01-2000';  
COMMENT ON TABLE "avion" IS '';
```

Actualizamos los valores de la tabla avión con los valores que tenía la tabla hangar

```
UPDATE avion a  
SET id_hangar= h.id_hangar, capacidad_hangar = h.capacidad, ubicacion = h.ubicacion  
FROM hangar h  
WHERE a.id_hangar= h.id_hangar;
```

Hacemos lo mismo para la tabla guarda

```
UPDATE avion a  
SET fecha_ingreso= g.fecha_ingreso,  
fecha_salida= g.fecha_salida  
FROM guarda g  
WHERE a.id_avion= g.id_avion;
```

Eliminamos las claves foráneas de la tabla guarda

```
ALTER TABLE "guarda"  
DROP CONSTRAINT "guarda_id_hangar_fkey"  
  
ALTER TABLE "guarda"  
DROP CONSTRAINT "guarda_id_avion_fkey"
```

Borramos las vistas "vista_aviones_hangar" y "vista_claveshangar"

```
DROP VIEW "vista_aviones_hangar"  
DROP VIEW "vista_claveshangar"  
DROP VIEW "vista_avion_propietario_hangar"
```

Borramos la tabla guarda

```
DROP TABLE "guarda";
```

Cambiamos la clave de avión

```
ALTER TABLE "avion"  
ADD CONSTRAINT "avion_id_avion_id_hangar_fecha_ingreso" PRIMARY KEY ("id_avion", "id_hangar",  
"fecha_ingreso"),  
DROP CONSTRAINT "avion_pkey";
```

Agregamos 2 columnas a la relación alquiler (los nuevos elementos de la clave primaria de avion)

```
ALTER TABLE "alquiler"  
ADD "id_avion" integer NOT NULL DEFAULT '0',  
ADD "fecha_ingreso" date NOT NULL DEFAULT '01-01-2000';  
COMMENT ON TABLE "alquiler" IS ";
```

Modificamos la clave foránea de la relación alquiler

```
ALTER TABLE "alquiler"  
DROP CONSTRAINT "alquiler_dni_fkey",  
ADD FOREIGN KEY ("id_hangar", "fecha_ingreso", "id_avion") REFERENCES "avion" ("id_hangar",  
"fecha_ingreso", "id_avion") ON DELETE RESTRICT ON UPDATE CASCADE
```

Agregamos 2 columnas a la tabla mantenimiento (los nuevos elementos de la clave primaria de avion)

```
ALTER TABLE "mantenimiento"  
ADD "id_hangar" integer NOT NULL DEFAULT '0',  
ADD "fecha_ingreso" date NOT NULL DEFAULT '01-01-2000';  
COMMENT ON TABLE "mantenimiento" IS ";
```

Actualizamos los valores de dichas columnas en la tabla mantenimiento

```
UPDATE mantenimiento m  
SET  
fecha_ingreso= a.fecha_ingreso,  
id_hangar= a.id_hangar  
FROM avion a  
WHERE m.id_avion=a.id_avion;
```

Actualizamos las claves foráneas de mantenimiento

```
ALTER TABLE "mantenimiento"  
ADD FOREIGN KEY ("id_avion", "id_hangar", "fecha_ingreso") REFERENCES "avion" ("id_avion",  
"id_hangar", "fecha_ingreso") ON DELETE SET NULL ON UPDATE CASCADE
```


Por ultimo actualizamos el nombre de la tabla avion

```
ALTER TABLE "avion" RENAME TO "avion_en_hangar";  
COMMENT ON TABLE "avion_en_hangar" IS ";
```

Tiempos luego de desnormalizar

```
EXPLAIN ANALYZE  
SELECT a.id_avion, a.tipo, a.id_propietario, a.peso, a.capacidad, a.fecha_ingreso, a.fecha_salida, a.id_hangar,  
a.ubicacion  
FROM avion a  
WHERE a.fecha_ingreso >= DATE '2025-01-01'  
AND a.fecha_ingreso < DATE '2026-01-01'  
AND a.ubicacion ILIKE 'Zona Sur';
```

QUERY PLAN

Seq Scan on avion a (cost=0.00..3619.09 rows=49637 width=43) (actual time=0.743..22.130 rows=50006 loops=1)

Filter: ((fecha_ingreso >= '2025-01-01'::date) AND (fecha_ingreso < '2026-01-01'::date) AND ((ubicacion)::text ~* 'Zona Sur'::text))

Rows Removed by Filter: 49999

Planning time: 0.235 ms

Execution time: 23.343 ms

Podemos observar que hay una mejora muy considerable a la hora de realizar la consulta. Esto sucede debido a que una única tabla tiene todos los datos necesarios para realizar la comparación.

Tiempo BD normalizada	Tiempo BD desnormalizada
Execution time: 81.157 ms	Execution time: 23.343 ms

Al comparar los tiempos de ejecución queda demostrado que el uso de la desnormalización mejora notablemente el rendimiento en una base de datos, en especial sobre consultas que requieren filtros sobre un gran volumen de datos.

Participaciones TPO2

Grupal

Investigación de preguntas teóricas y elección de índices para las relaciones. Luego se realizó en conjunto el punto de la desnormalización.

Ruiz Milagros

Se encargó de realizar la tabla comparativa de índices y realizar las consultas para crear los índices.

Ferreya Agustín

Se encargó de realizar el punto 4, junto con la comparación de los tiempos sobre los índices creados.

Bibliografía

- [1] <https://www.ibm.com/docs/es/db2/12.1.x?topic=objects-indexes>
- [2] <https://www.linkedin.com/advice/0/what-database-index-how-does-improve-performance?lang=es&originalSubdomain=es>
- [3] <https://medium.com/@khouloud.haddad/drawbacks-of-database-indexes-eb4839b88ac1>
- [4] <https://www.postgresql.org/docs/12/indexes-types.html>
- [5] <https://www.postgresql.org/docs/12/indexes-types.html>
- [6] <https://www.postgresql.org/docs/current/hash-index.html>
- [7] <https://www.postgresql.org/docs/current/sql-vacuum.html>
- [8] <https://www.postgresql.org/docs/current/sql-analyze.html>
- [9] <https://www.postgresql.org/docs/current/sql-explain.html>